

Mini Game Hub

SSL Project Report — 2026

Samhitha Poladi (25B1088)
Riddhima Singh (25B1068)

April 2026

Contents

1	External Libraries Used and Their Purpose	3
1.1	Pygame	3
1.2	NumPy	3
1.3	Matplotlib	3
1.4	Bash Utilities	3
1.5	Python Dependencies	3
2	Description of All Features Implemented	5
2.1	User Authentication System	5
2.2	Game Menu (game.py)	5
2.3	Five Playable Games	5
2.4	Common In-Game UI Elements	5
2.5	Game History Recording	6
2.6	Leaderboard (leaderboard.sh)	6
2.7	Data Visualisations	6
3	Explanation of Code Logic	7
3.1	Generic Game Class (game.py)	7
3.2	Authentication System (main.sh)	7
3.3	Tic-Tac-Toe (tictactoe.py)	8
3.4	Connect Four (connect4.py)	8
3.5	Othello (othello.py)	8
3.6	Battleship (battleship.py)	9
3.7	Pong (pong.py)	9
3.8	Leaderboard (leaderboard.sh)	10
4	Hurdles Faced and How We Overcame Them	11
4.1	Cross-File Module Imports and Asset Paths	11
4.2	Othello Valid-Move Algorithm	11
4.3	Bash and Python I/O Separation	11
4.4	Carriage Returns in CSV	11
4.5	Battleship Turn Switching on Hits	11
4.6	Pygame Window Persistence Between Games	11

5	What We Would Do Differently With 10 More Hours	12
5.1	Save Good Games	12
5.2	Have an Option for Placing Ships in Battleship	12
5.3	Animated UI and Sound Effects	12
5.4	Responsive Layout	12
6	Bibliography	13

1 External Libraries Used and Their Purpose

The project uses several well-established external libraries, each serving a specific role in the system.

Library	Language	Purpose
pygame	Python	GUI rendering, event handling, game windows
numpy	Python	Board representation, win-checking logic
matplotlib	Python	Data visualisations (pie chart, bar chart)
hashlib / shasum	Python / Bash	Password hashing (SHA-256)
os	Python	File path management across platforms
time	Python	End-game delay and timing logic
random	Python	Random ship placement in Battleship
sys	Python	Clean process exit in Pong
awk / grep / sort	Bash	Text processing in leaderboard script

1.1 Pygame

Pygame is a cross-platform Python library for building multimedia applications. In this project it manages the display window, captures mouse and keyboard events, renders shapes and text, and handles the game loop at 60 FPS.

1.2 NumPy

NumPy provides the `ndarray` data structure used to represent every game board. Its vectorised Boolean operations allow efficient win detection without nested Python loops, keeping game-state checks fast even on large boards (e.g. the 10×10 Tic-Tac-Toe grid).

1.3 Matplotlib

Matplotlib generates static charts from the game history CSV file: a pie chart of the most-played games and a bar chart of the top five players by win count.

1.4 Bash Utilities

Standard POSIX tools (`awk`, `grep`, `cut`, `sort`, `uniq`) are used in `leaderboard.sh` to parse and aggregate the CSV history file without requiring any additional external packages.

1.5 Python Dependencies

The required Python dependencies are listed in the `requirements.txt` file included in the project directory.

```
pygame
numpy
matplotlib
```

These can be installed using:

```
pip install -r requirements.txt
```

2 Description of All Features Implemented

2.1 User Authentication System

The authentication system, implemented entirely in Bash (`main.sh`), supports:

- **Registration** — new users choose a username and password; the password is hashed with SHA-256 via `shasum -a 256` before being appended to `users.tsv`.
- **Login** — returning users are authenticated by re-hashing their entered password and comparing it to the stored hash.
- **Two-player enforcement** — the system ensures Player 2 must be a different account from Player 1, looping until a distinct user is authenticated.
- **Persistent storage** — credentials are stored in a tab-separated file (`users.tsv`) that persists across sessions.

2.2 Game Menu (`game.py`)

After authentication, a Pygame-based menu allows the two authenticated players to select from the available games. The menu launches the chosen game module and records the result (winner, loser, game name) to `history.csv` upon completion.

2.3 Five Playable Games

1. **Tic-Tac-Toe** — 10×10 board; five-in-a-row win condition; Player 1 plays circles (O), Player 2 plays crosses (X).
2. **Connect Four** — 7×7 board; gravity-based token drop; four-in-a-row win in any direction.
3. **Othello (Reversi)** — 8×8 board; standard disc-flipping rules; winner determined by piece count at game end.
4. **Battleship** — 10×7 dual-grid board; randomised ship placement; players fire alternately; all enemy ships sunk wins.
5. **Pong** — real-time paddle-and-ball game; first to 5 points wins; ball angle varies with paddle contact point.

2.4 Common In-Game UI Elements

All games share:

- Themed background images loaded from an `assets/` directory.
- A consistent retro font (`PressStart2P-Regular.ttf`).
- A **BACK** button (returns to menu without recording a result) and a **RESET** button (restarts the current game).

- A winner/draw pop-up overlay displayed 0.75 s after the game ends, auto-dismissed after 2.5 s.
- Mouse-hover colour change (white → yellow) on interactive buttons.

2.5 Game History Recording

Every completed game appends a row to `history.csv` containing the winner's name, the loser's name, and the game title. This flat file acts as the single source of truth for all leaderboard and analytics calculations.

2.6 Leaderboard (`leaderboard.sh`)

The leaderboard is a Bash script that:

- Reads `history.csv` and lists stats (wins, losses, win/loss ratio) per player, broken down by game.
- Displays the game passed as an argument first, followed by all other games found in the history.
- Supports three sort modes: by wins (1), by losses (2), or by win/loss ratio (3).
- Shows UNDEFEATED for players with no losses.

2.7 Data Visualisations

Two Matplotlib charts are generated from `history.csv`:

- **Most Played Games** — a pie chart showing the percentage of matches played per game.
- **Top 5 Players** — a bar chart ranking the five players with the highest win counts.

3 Explanation of Code Logic

3.1 Generic Game Class (game.py)

All five games inherit from a single base class, `games_class`, which encapsulates shared state and behaviour:

Listing 1: Core base class attributes

```
class games_class:
    def __init__(self, player1, player2, board=None):
        self.player1 = player1    # authenticated username
        self.player2 = player2
        self.board = board        # numpy ndarray (game-specific shape)
        self.active = 1           # whose turn (1 or 2)
        self.n_active = 2         # the other player
        self.winner = 0           # 0 = no winner yet
        self.win_line = None      # used by Tic-Tac-Toe for line drawing

    def switch(self):
        self.active, self.n_active = self.n_active, self.active
```

Each subclass must implement three methods:

1. `win_check()` — inspects the board and returns the winning player identifier or 0.
2. `show(screen, mouse_pos)` — renders the background, labels, and UI buttons for a single frame.
3. `execution()` — contains the main game loop; returns the winner (or 0 for draw/back).

This structure eliminates code duplication and ensures that adding a new game requires only implementing the three game-specific methods.

3.2 Authentication System (main.sh)

Listing 2: Password hashing and credential lookup

```
hash_password(){
    echo "$1" | shasum -a 256 | awk '{print_$1}'
}

authenticate_user(){
    read -p "Username:_" username
    stored=$(grep "^$username" "$users")
    if [[ -n $stored ]]; then
        read -sp "Password:_" password
        hashed=$(hash_password "$password")
        stored_hash=$(echo "$stored" | cut -f2)
        [[ $hashed == $stored_hash ]] && echo "$username"
    fi
}
```

The function outputs the authenticated username to stdout, while all prompts and messages go to stderr (&2), allowing the calling code to capture only the clean username via command substitution: `user1=$(authenticate_user 1)`.

3.3 Tic-Tac-Toe (tictactoe.py)

The board is a 10×10 NumPy array. Win detection checks for five-in-a-row by constructing Boolean truth matrices and combining them with bitwise AND. If we have five-in a row then that position in the final matrix will have true value and rest all will have false. Since this returns the position from where the five in a row sequence begins we can use it to extract the exact position where the win happens and a win line is drawn:

Listing 3: Vectorised horizontal win check

```
t = self.board == self.active # Boolean matrix
h = (t[:, 0:5] & t[:, 1:6] & t[:, 2:7]
      & t[:, 3:8] & t[:, 4:9])
if np.any(h):
    posn = np.where(h == 1)
    # record win_line endpoints for drawing
    return self.active
```

The same pattern repeats for vertical and both diagonal directions. A hover effect previews each player's symbol in a dimmed colour before they click.

3.4 Connect Four (connect4.py)

The 7×7 board uses values 1 (Player 1), 2 (Player 2), and -1 (winning tokens highlighted in orange). Gravity is handled by scanning a column from the bottom row upward until an empty cell is found:

Listing 4: Gravity: finding the lowest empty row

```
def available_row(self, column):
    for i in range(6, -1, -1):
        if not self.occ(i, column):
            return i
    return -1 # column is full
```

Win detection uses the same sliding Boolean window technique as Tic-Tac-Toe, but for windows of length 4 in all four orientations. Winning cells are set to -1 so they render in a distinct highlight colour.

3.5 Othello (othello.py)

Othello is the most algorithmically complex game. The core method is `finding_valid(box, check=False)`:

1. It inspects the 3×3 neighbourhood around the clicked cell to find adjacent opponent pieces.

2. For each direction containing an opponent piece, it walks along that direction until it hits a boundary, an empty cell (invalid), or the active player's own piece (valid — flip all pieces in between).
3. When `check=True`, it returns a Boolean immediately without modifying the board, used to pre-highlight valid moves and detect when no valid moves remain.

When a player has no valid moves the turn is skipped; if neither player can move the game ends and the winner is determined by piece count.

3.6 Battleship (battleship.py)

Each player has an independent 10×7 NumPy board for ship positions and a separate hits grid. Ships are placed randomly with collision detection:

Listing 5: Ship placement with collision detection

```
def place_ships(self, board):
    for ship_size in [5, 4, 3, 3, 2]:
        placed = False
        while not placed:
            orientation = random.choice(["H", "V"])
            if orientation == "H":
                r = random.randint(0, 9)
                c = random.randint(0, 7 - ship_size)
                if np.all(board[r, c:c+ship_size] == 0):
                    board[r, c:c+ship_size] = 1
                    placed = True
```

The win condition checks whether all opponent ship cells have been hit:

Listing 6: Win condition: all ships sunk

```
def win_check(self):
    if np.all((self.hits_p2 == 1) | (self.board_p2 == 0)):
        return self.player1
    elif np.all((self.hits_p1 == 1) | (self.board_p1 == 0)):
        return self.player2
    return 0
```

3.7 Pong (pong.py)

Ball-paddle collisions compute a reflection angle proportional to how far the ball strikes from the paddle centre. Ball speed increases by 5% on every hit and is capped at 15 pixels/frame:

Listing 7: Angle and speed modulation on paddle hit

```
if self.ball.collidect(self.paddle1):
    self.ball_speed_x = abs(self.ball_speed_x)
    offset = ((self.ball.centery - self.paddle1.centery)
              / (self.paddle1.height // 2))
    self.ball_speed_y = offset * 6
```

```
self.ball_speed_x *= 1.05
self.ball_speed_y *= 1.05
max_speed = 15
self.ball_speed_x = max(-max_speed, min(max_speed, self.ball_speed_x))
self.ball_speed_y = max(-max_speed, min(max_speed, self.ball_speed_y))
```

Player 1 uses W/S keys; Player 2 uses UP/DOWN arrow keys. First to 5 points wins.

3.8 Leaderboard (leaderboard.sh)

The script iterates over each unique game found in `history.csv`. For each game it uses `awk` to count wins (rows where column 1 matches the player) and losses (rows where column 2 matches). The ratio is computed with `awk` floating-point arithmetic. Players with zero losses are marked `UNDEFEATED` with a sentinel ratio of `999.00` so they always sort to the top.

4 Hurdles Faced and How We Overcame Them

4.1 Cross-File Module Imports and Asset Paths

Because each game file resides in a `games/` subdirectory while assets live in `assets/`, hard-coded relative paths broke when the working directory changed. We resolved this by computing absolute paths at runtime using `os.path.dirname(
file
)and os.path.join, making every import and image load independent of the invocation directory.`

4.2 Othello Valid-Move Algorithm

Correctly implementing Othello’s flip logic along all eight directions and handling board edge cases required several iterations. We refactored to use NumPy diagonal slicing (`np.diagonal`) and `np.argmin` to locate the first non-opponent piece along each ray, which simplified the code and eliminated boundary bugs.

4.3 Bash and Python I/O Separation

The `authenticate_user` function needed to return a username while also printing prompts to the terminal. Mixing `stdout` and `stderr` caused the captured output to include prompt strings. The fix was to redirect all human-readable messages to `&2` (`stderr`) so that only the clean username was emitted on `stdout` and captured by command substitution.

4.4 Carriage Returns in CSV

On some systems `history.csv` contained Windows-style `\r\n` line endings. The `awk` comparisons in `leaderboard.sh` failed silently because the game name field included a trailing `\r`. We added `sub(/\r$/, "", $3)` in every `awk` invocation to strip the carriage return before any comparison.

4.5 Battleship Turn Switching on Hits

The original design switched turns after every click. In standard Battleship the shooter continues on a successful hit. We added a conditional so `switch()` is called only when the fired cell does not contain a ship, giving the correct “keep going on a hit” behaviour.

4.6 Pygame Window Persistence Between Games

Returning from one game to the menu while a Pygame display was still active caused the next game to open a second window. We ensured each game’s `execution()` method returns cleanly and that the menu reclaims the single display via `pygame.display.quit()` avoiding duplicate windows.

5 What We Would Do Differently With 10 More Hours

5.1 Save Good Games

Add an option to record moves for Tic-tac-toe, Othello and Connect 4 and save these games to access later.

5.2 Have an Option for Placing Ships in Battleship

To stay true to the arcade theme we had the ships be randomised as players would have to look away as the other one places ships. We thought of adding an option to make this possible, upto the choice of the players.

5.3 Animated UI and Sound Effects

Adding Pygame sound effects (coin drops for Connect Four, explosion sounds for Battleship) and smooth animations (token drop physics, disc flip for Othello) would significantly improve the player experience with modest additional code.

5.4 Responsive Layout

All coordinates are currently hard-coded for a 900×750 pixel window. Abstracting layout to proportional values would allow the games to scale to different screen sizes without manual recalculation.

6 Bibliography

1. Pygame Community. *Pygame Documentation*. <https://www.pygame.org/docs/>
2. Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
3. Hunter, J.D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
4. Python Software Foundation. *os -- Miscellaneous operating system interfaces*. <https://docs.python.org/3/library/os.html>
5. Python Software Foundation. *hashlib -- Secure hashes and message digests*. <https://docs.python.org/3/library/hashlib.html>
6. GNU Project. *Bash Reference Manual*. <https://www.gnu.org/software/bash/manual/bash.html>
7. The Open Group. *awk -- pattern scanning and processing language (POSIX)*. <https://pubs.opengroup.org/onlinepubs/9699919799/utilities/awk.html>
8. World Othello Federation. *Official Othello Rules*. <https://www.worldothello.org/about/about-othello/othello-rules/official-rules/english>
9. Google Fonts. *Press Start 2P*. <https://fonts.google.com/specimen/Press+Start+2P>
10. NumPy Developers. *numpy.diagonal reference*. <https://numpy.org/doc/stable/reference/generated/numpy.diagonal.html>